Solution Architecture

Date	03 October 2026
Team ID	2
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Solution Architecture Diagram:

FitBuddy follows a layered architecture: a server-rendered presentation layer, a FastAPI application layer containing authentication, core logic and the Gemini integration, and a SQLite data layer.

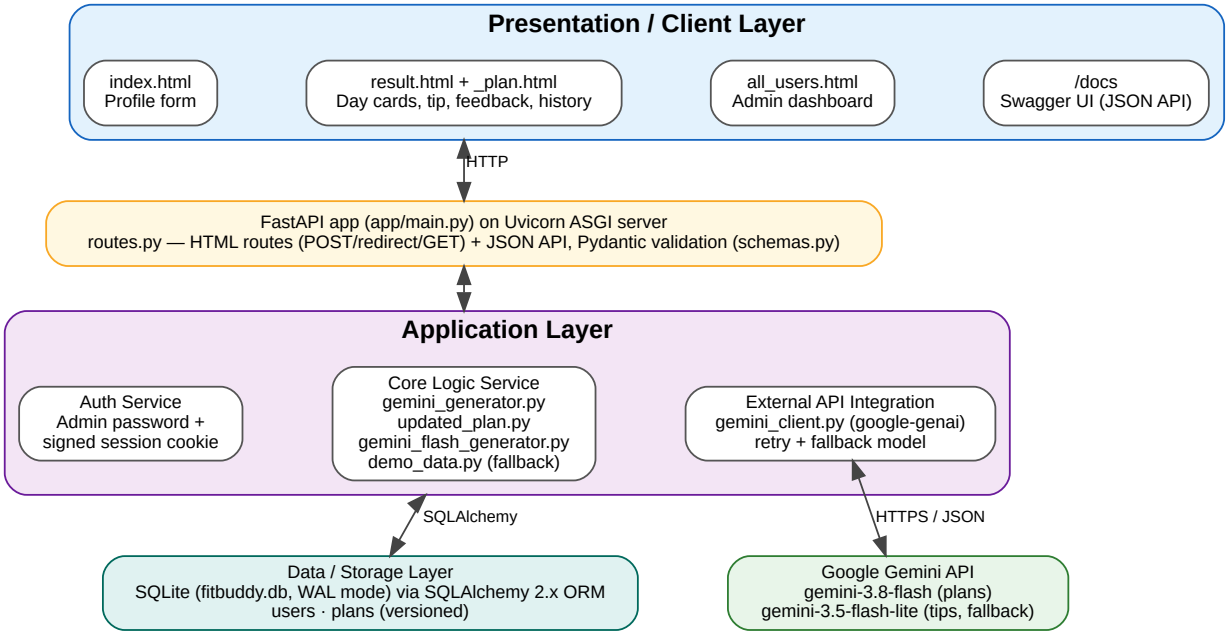


Figure 1: FitBuddy solution architecture

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Profile form, result page with day cards / tip / feedback / version history, admin dashboard and login, friendly error page, Swagger UI.	HTML5, CSS3, Jinja2 (index.html, result.html, _plan.html, all_users.html, admin_login.html, error.html)
API Gateway (Web server & routing)	Receives HTTP requests, validates input, routes to services, returns HTML (POST/redirect/GET) or JSON with proper status codes (422, 404, 502, 401).	FastAPI (main.py, routes.py), Uvicorn, Pydantic (schemas.py)

Component Name	Description / Role in Architecture	Technologies Used
Auth Service	Checks the admin password and issues a signed session cookie for dashboard and delete routes.	itsdangerous signed cookies, ADMIN_PASSWORD / SECRET_KEY in .env
Core Logic Service	Builds prompts from age, weight, goal, intensity and experience; validates the 7-day plan; revises the latest version from delimited feedback; generates tips; demo-mode fallback.	gemini_generator.py, updated_plan.py, gemini_flash_generator.py, demo_data.py
External API Integration	Single shared Gemini client with structured JSON output, timeout, one retry and automatic fallback model; rejected keys fail fast.	google-genai SDK (gemini_client.py), Gemini API
Database	Stores users and versioned plans; cascade delete; WAL mode for concurrent writes.	SQLite, SQLAlchemy 2.x (database.py)

Request flow (generate plan)

Browser submits the form → routes.py validates with Pydantic → gemini_generator.py builds the prompt → gemini_client.py calls Gemini with the plan JSON schema (retry, then fallback model) → plan validated → tip generated → user and plan v1 saved → redirect to /plan/{user_id} which renders the day cards.

Entity-Relationship (ER) Diagram

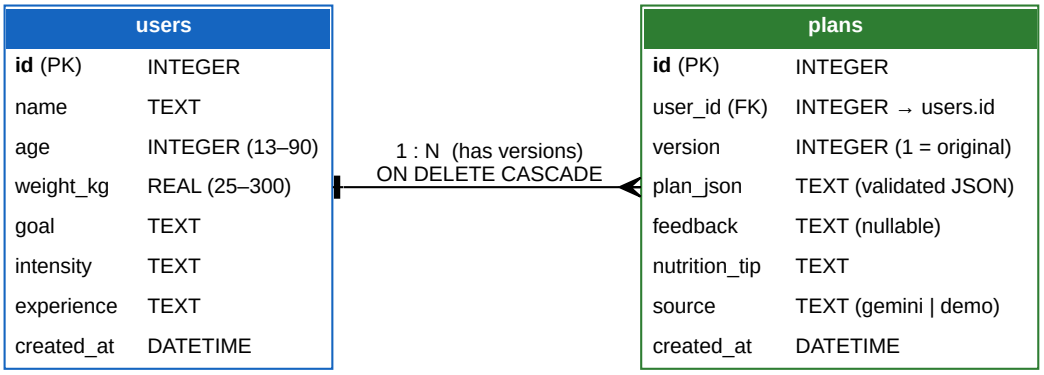


Figure 2: FitBuddy ER diagram — one user has many plan versions

One **user** has many **plans**. Version 1 is the original; each feedback submission adds a new version row instead of overwriting. Deleting a user removes all of their plans (ON DELETE CASCADE).